ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

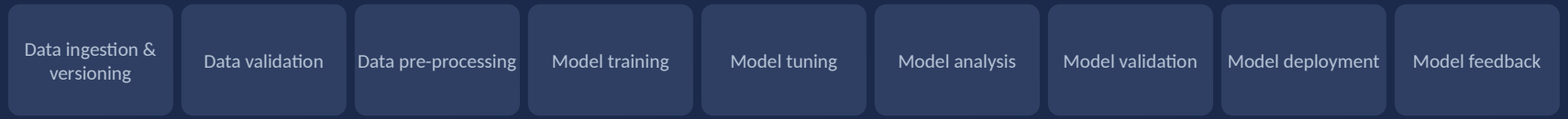
MLOps: Machine Learning Operations

Session 5

CI/CD & Deployment Strategies

Juan Manuel Ramírez García · jramirezg@faculty.ie.edu

Live online session · Saturday 17 October 2026 · 90 minutes



Where we are: what Forum 2 said, and what today adds

From Forum 2: 29 of you posted

What you said

- *"Build once, promote the same artefact."* — **Sigursteinn**
- *"STOP if critical, warning if small."* — **Anas**
- **Misconception of the week:** a validation that stops on every failed check stops every run. Stop on what is broken; flag what is known.

By the end of this session you can...

Draw a CI/CD flow for a model

And say which tests run on code and which on data.

Choose a rollout strategy

Canary, blue-green, shadow or A/B, for a given business constraint.

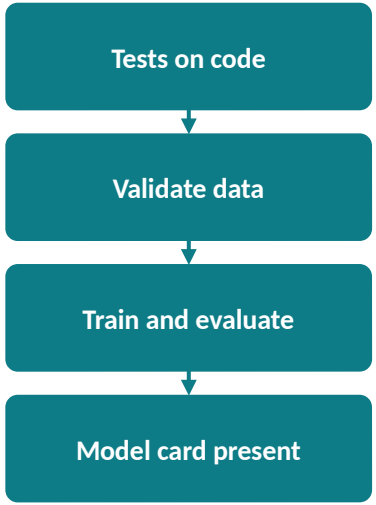
Minute	Block
0–8	Recap, outcomes, "what did the tick check?", ML vs software
8–16	CI, CD and the third letter: CT; the three levels
16–35	Tests for code and data (+ 6-min demo) · the artefact · serving modes · containers (+ poll, Demo 2)
35–46	Rollout strategies: basic, blue-green, shadow, canary, A/B
46–51	Break
51–58	Serving at scale: scaling, Kubernetes, serverless
58–76	Activity: choose a rollout for a bank
76–86	Debrief
86–90	Three things, your forum, your reading

What did the green tick check?

What students usually guess

- "That the code works"
- "That the model is accurate"
- "That nothing broke"
- "That someone reviewed it"

What actually runs on every push



the validation rules behave as specified

the training extract passes the checks

the model trains end to end on the extract

an owner is written down

Nobody ran it. It ran because someone pushed a change. Today is about that sentence.

Why ML systems need their own version of CI/CD

Team
Data scientists and researchers, not production engineers.

Development
Experimental: many features, algorithms, settings. Tracking what worked is the hard part.

Testing
Unit and integration tests are not enough: data validation, model quality, model validation.

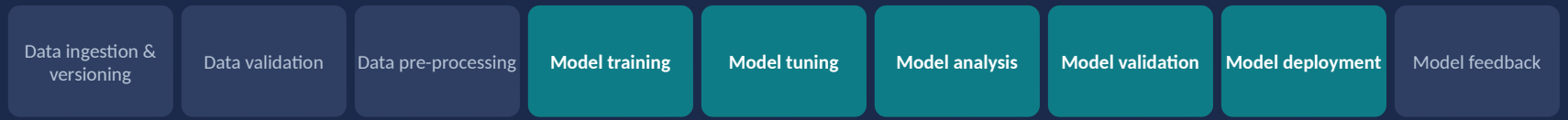
Deployment
Not one service: a multi-step pipeline that retrains and redeploys another service.

Production
Models decay as data changes. Monitor, alert, roll back.

Source: Google Cloud, "MLOps: continuous delivery and automation pipelines in machine learning"

CI, CD, and the third letter: CT

From a manual process to a pipeline that retrains itself



The lifecycle we follow through the course · highlighted: where this section sits

CI, CD, and CT: what each one means when the product is a model

Continuous Integration

- Every change is merged and checked automatically
- For ML: tests on code, and on data, schemas and models

Continuous Delivery

- A verified build reaches production automatically
- For ML: what is delivered is a pipeline that deploys a prediction service

Continuous Training

- New, unique to ML
- Retrain and serve the model automatically when new data arrives or performance drops

The demo repository has CI (tests on code and data) and the start of CD (a training dry-run). CT would be a trigger that runs it on new data and registers the result.

Three levels of maturity: manual, automated pipeline, automated CI/CD

Level 0 · Manual

- Notebook → model file → handed to engineers
- Infrequent releases, no CI/CD
- No monitoring; nobody notices decay

Level 1 · Automated pipeline

- New data triggers validate → train → evaluate → deploy
- Continuous training of the model
- The pipeline itself is still deployed by hand

Level 2 · CI/CD for the pipeline

- Changes to the pipeline are built, tested and deployed automatically
- Data scientists try ideas safely and often
- Registry, metadata, triggers everywhere

Where was NorthRetail? Level 0. Where is IberBank after the Friday hotfix? Level 0, with a laptop. Most companies live here; the aim is Level 1.

Source: Google Cloud MLOps whitepaper

The automated pipeline, stage by stage, and what starts it



Try algorithms and features; steps orchestrated

Code pushed → built → tested (code and data)

Pipeline deployed to the target environment

Runs on new data → trained model → registry

Model served as a prediction service

Performance statistics → next trigger

What starts a run

A push
Someone changed code.

A schedule
Every night at 2 a.m.

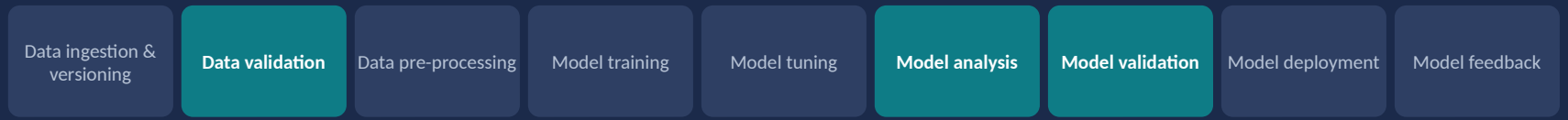
New data
A new extract landed.

A performance signal
Accuracy fell below a line (S7).

Level 1 is the middle of this drawing running on its own. Level 2 is the left half running on its own too.

Tests: for code, for data, for models

What a pipeline checks before anything reaches production



The lifecycle we follow through the course · highlighted: where this section sits

The ML test pyramid: code tests, data tests, model tests

Model tests · every candidate

Minimum metric on a fixed evaluation set · not worse than production · fairness by segment · latency and memory within budget

Data tests · every new extract

Schema (columns, types) · ranges and allowed values · freshness · volume · no leakage

Code tests · every push

Unit tests: does each function do what it says? · Integration tests: do the steps fit together? · seconds, automatic

Cheap and frequent at the bottom, expensive and rare at the top. A failure at any layer stops the pipeline: nothing reaches production without passing all three.

What a pipeline run looks like

A green run

Someone pushed a change; four steps ran; every one passed. The change may go on.

A red run

A new data extract arrived with a feature 70% empty. The pipeline stopped at "Validate data". Nothing was trained; nothing was deployed.

A record

Every run is kept: who triggered it, which commit, which step failed, the full log. Auditors like this.



Live demo · 6 minutes

GitHub Actions on the demo repository: the green run on main, the red run on the broken extract, and the log line that stopped it. No commands.

What you just saw

It ran by itself

A push is the trigger. Nobody has to remember to run the tests, and nobody can forget to.

It stopped at the data test

Before training, before deployment, with a message a human can read. That is what "validation is a control" means.

It left a record

Commit, author, time, every log line. When the regulator asks "what did you check before releasing?", this is the answer.

The pipeline decided the change was safe. Next: what exactly gets shipped, and how it reaches customers.

What gets shipped: the ML artefact is more than the model file

Code

model and pre-processing

Configuration

hyperparameters, settings

Data reference

training and validation data: version, hash

The model

trained, in runnable form

The environment

libraries with exact versions, system dependencies

Documentation

the model card: owner, metrics, retrain trigger

Test scenarios

code and data to re-test it anywhere

Shipping only the .pkl file is how the laptop-to-production failures of Session 3 happen.

Adapted from Treveil et al., Introducing MLOps (O'Reilly, 2020)

Four ways a model serves: real-time, batch, stream, edge

Real-time

One record, on request, through an endpoint. Milliseconds. The fraud model.

Operational concern: Latency, availability, rollout without downtime

Batch

Whole datasets on a schedule. The churn list every Monday at 6 a.m.

Operational concern: Throughput, cost, a missed run

Stream

Continuous scoring of events from a stream, results back to a stream. The velocity features.

Operational concern: Ordering, back-pressure, state

Edge

On the device or a local server: smallest latency, works offline. A camera on a factory line.

Operational concern: Updating models on thousands of devices

Today's activity is about real-time serving; batch and stream come back in Session 7 (monitoring).

The artefact travels in a box: containers

Why a box

- A model version is code + an exact environment: libraries, versions, system dependencies
- Deployment must rebuild that environment reliably on the target machine
- One machine runs many models with incompatible needs

Container vs virtual machine

- A VM carries a whole operating system: heavy, slow to start
- A container shares the host's OS and carries only what the app needs: light, seconds to start
- Docker builds and runs containers; Kubernetes runs thousands (next block)

Chat poll (2 min): "A model that passed all automated tests can be released to 100% of customers at once." AGREE or DISAGREE, one line why.

Demo: from the registry to a live endpoint, in four steps

1 · A model in the registry

Version 1 of churn-model carries the label production. We ship exactly that version.

churn-model · registered model

Version 1	**@production**
Version 2	@challenger
Version 3	

`docker images churn-model`

2 · A Dockerfile

The recipe for the box: same Python, exact libraries, the model, a web server.

```
# same Python as training
FROM python:3.10.12-slim
# the exact libraries
RUN pip install …
# the model itself
COPY model /opt/ml/model
# a prediction server
CMD mlflow models serve
```

`cat registry_box/Dockerfile`

3 · A container running

The box switched on: a small web service, waiting for requests on port 5002.

container · churn-prod

- web server · port 8080
- model v1 (production)
- Python 3.10 + exact libraries

● **running · :5002/ping → 200**

`docker run … · curl …/ping`

4 · Scoring two customers

Two customers sent over the network; the production model answers in milliseconds.

Loyal customer
60 months · €45 · 0 tickets → **0**

At-risk customer
3 months · €110 · 5 tickets → **1**

`{"predictions": [0, 1]}`

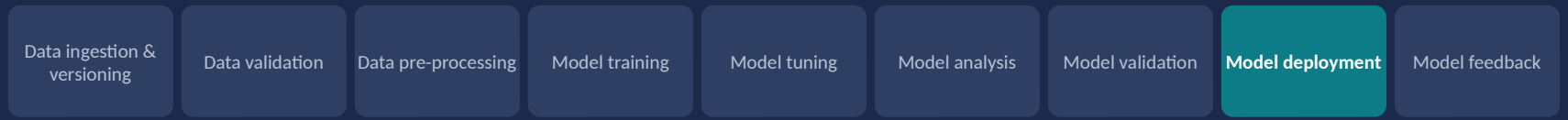
0 = stays **1** = likely to leave

`curl -X POST …/invocations`

This is what "deployed" means: a registered version, packed with its exact environment, answering requests. Rollback = stop this box and start the previous one, which still exists.

Rollout: reaching customers without downtime

Basic, blue-green, shadow, canary, and the A/B test that decides



The lifecycle we follow through the course · highlighted: where this section sits

First, the requirement: "zero downtime", and the strategy that ignores it

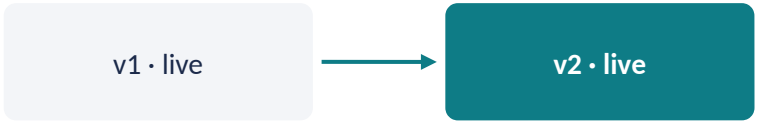
Zero downtime means

At no moment is the service unavailable or answering wrongly because of the release. Customers never notice. For card payments this is not optional.

Rollback means

Returning to the previous artefact: model, code and configuration. It must still exist, and be deployable in minutes, not days.

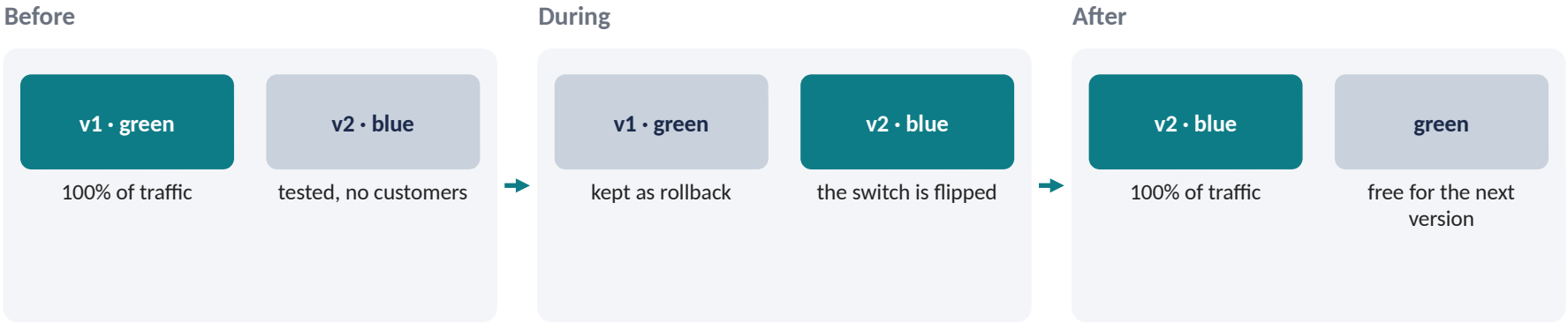
Basic deployment: replace everything at once



Simple, fast, cheap, risky: for a moment everyone gets the untested behaviour, or nothing. Fine for non-critical services, lower environments, off-hours.

Adapted from Treveil et al., Introducing MLOps (O'Reilly, 2020)

Blue-green: two identical environments, and a switch



Pros

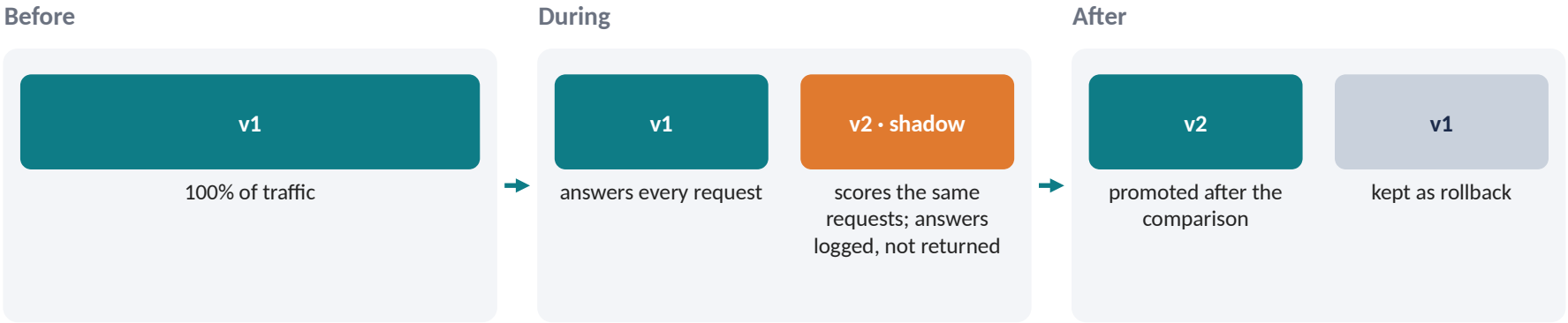
- Simple, well understood, easy to implement
- Rollback = flip the switch back
- Zero downtime if the switch is instant

Cons

- Two full production environments
- Realistic test traffic for blue is not free
- Keeping both truly identical is hard

Adapted from Treveil et al., Introducing MLOps (O'Reilly, 2020)

Shadow: the new model scores everything, and nobody sees its answers



Pros

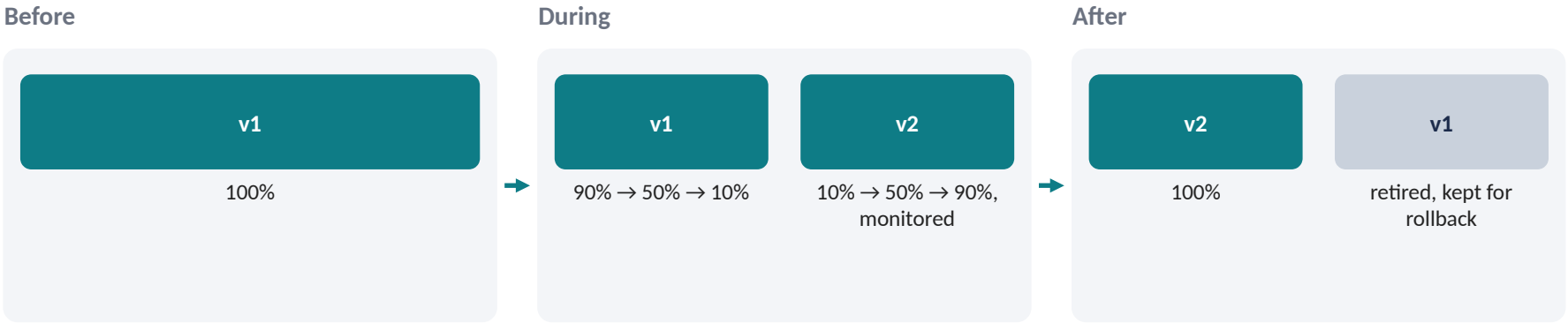
- v2 sees real data with zero customer risk
- Rollback = switch off the shadow
- Proof of equivalence before anyone is affected

Cons

- Two production environments at once
- All traffic to both: twice the compute
- You learn what v2 would say, not how customers react

Adapted from Treveil et al., Introducing MLOps (O'Reilly, 2020)

Canary: a small slice of customers first, then more, then everyone



Pros

- No second full environment: cheaper
- Risk bounded by the slice
- The lowest-risk way to reach real customers

Cons

- Two behaviours in production at once
- The slice must be big enough to see, small enough to limit damage
- Needs monitoring good enough to decide

Adapted from Treveil et al., Introducing MLOps (O'Reilly, 2020)

A/B testing is not a canary: it is an experiment with a decision rule

Canary asks: is v2 safe?

- Watches errors, latency, obvious wrong answers
- Grows the slice when nothing breaks
- A release mechanism

A/B asks: is v2 better?

- Random split into two comparable groups
- One metric, decided in advance
- A decision rule; an experiment

The three ingredients of an A/B test

A metric

Fraud caught at a fixed false-positive rate; conversion; revenue per user.

A threshold

"v2 wins if it catches 2 points more fraud with the same false positives."

Enough traffic

So the difference is not luck. More than a busy afternoon; statisticians set the number.

The scorecard: which strategy for which constraint

	Basic	Blue-green	Shadow	Canary	A/B test
Zero downtime	No	Yes, if the switch is instant	Yes	Yes	Yes
Extra cost	None	A second full environment	A second environment; double compute	Small	Small
Customer risk during rollout	Everyone	Everyone, after the switch	None	A slice	Half
What you learn	Nothing until it breaks	What QA found	What v2 would have said	Whether v2 is safe	Whether v2 is better
Rollback	Redeploy v1	Flip the switch back	Switch off the shadow	Shrink the slice to 0%	Stop the test

They combine: shadow first (proof of equivalence for the regulator), then a canary, with an A/B decision rule inside it. That combination is today's activity.

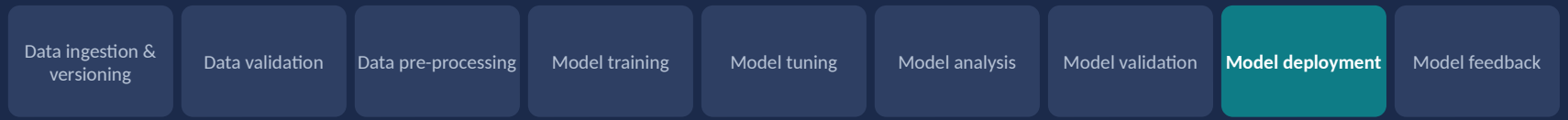
ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Break · 5 minutes

Back at minute 51. While you wait: what happens to your bank's app when 10× the usual customers log in at once?

Serving at scale: servers, containers, serverless

What holds the model up when the traffic arrives



The lifecycle we follow through the course · highlighted: where this section sits

Scaling: more power, more machines, or both, automatically

Vertical: a bigger machine

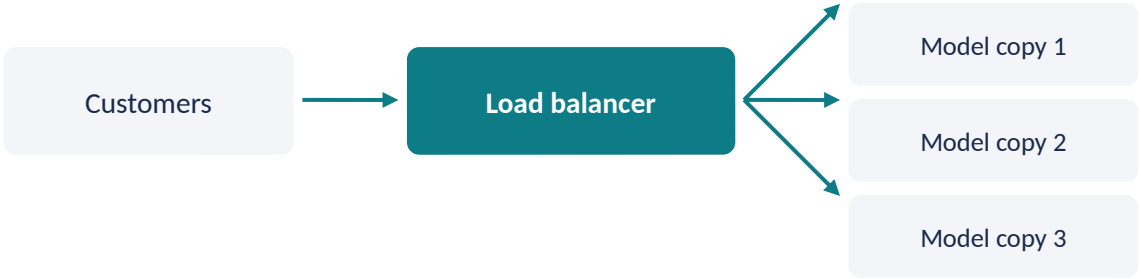
- Add CPU, memory, GPU
- The first reflex; simple
- Hits a ceiling, and a price

Horizontal: more machines

- Replicate the service; a load balancer shares requests
- No ceiling
- More moving parts

Autoscaling: on its own

- Scheduled: we know the peak is coming
- Dynamic: CPU, requests or latency cross a line → add machines; fall → remove



Elastic: copies are added when the metric rises and removed when it falls, so you pay for the peak only during the peak.

Kubernetes in one slide: the manager of the boxes

Without it

- Start the right containers at the right time
- Make them talk to each other
- Handle storage; replace failed containers and machines
- By hand, day and night

With Kubernetes

- Deploys, scales and manages containers automatically
- Load balancing, self-monitoring, restart on failure
- Storage orchestration
- Docker makes the box; Kubernetes runs thousands of boxes

Scalable: you can add machines. Elastic: the system adds and removes them by itself. A Kubernetes cluster in the cloud is both. Managed versions: every cloud; OpenShift for the enterprise.

Serverless: pay for the seconds the model runs, and nothing else

Run a web server

Flask, application servers. You own the machine, the scaling, the patches.

A cloud ML platform

SageMaker, Vertex AI, Azure ML, watsonx.ai host the model and the endpoint for you.

Serverless functions

AWS Lambda, Cloud Functions, Azure Functions: upload a function; it runs on demand.

How a serverless prediction works



Fits

Irregular or low traffic · simple models · no infrastructure team · pay per second.

Does not fit

Strict latency (cold starts) · heavy models · constant high traffic (cost) · the fraud model.

Activity · 15 minutes in breakout rooms: choose a rollout for a bank

Your task (15 min)

- Read the case: IberBank card-fraud model, v2 is validated
- Decide the rollout plan: which strategies, in which order, with what traffic slices, and why not the others
- Write the A/B decision rule: metric, threshold, enough traffic
- Define the rollback trigger: which number, watched by whom, stops the rollout
- Say who signs each step

How to work

- Rooms of 4–5: your project team
- Rollout card template on campus (or a whiteboard)
- Use today's vocabulary: the five strategies, the scorecard, the three A/B ingredients, the test pyramid
- Nominate a spokesperson: 2 minutes
- Bonus: one CI step you would add for this model

Output per room: one slide or whiteboard. Three rooms present in the debrief; everyone posts theirs on the forum.

The case: IberBank card-fraud model, version 2 (facts only)

- Version 1 has been in production since March: 40 million transactions a month, under 150 ms, three channels (point of sale, e-commerce, ATM). It blocks about 0.4% of transactions; roughly one in three blocks is a false positive.
- Version 2 passed validation last week: on the evaluation set it catches 9% more fraud at the same false-positive rate, and the fairness check by region passed. It uses two new velocity features from the streaming system.
- Constraint 1: zero downtime. Card payments run 24/7; a minute without scoring means every transaction is either blocked or waved through.
- Constraint 2: the regulator, after the March incident, requires proof on real traffic that v2 is at least as safe as v1 before any customer is affected by it.
- Constraint 3: the infrastructure budget allows a second production copy of the model for at most four weeks.
- Constraint 4: the fraud operations team checks by hand the transactions the model sends to its review queue: about 300 a day, its full capacity. More than that and reviews are skipped. Chargebacks (the ground truth) arrive up to 60 days late.

Rollout card

- Plan: strategies, order, slices, duration
- Why not the others
- A/B rule: metric · threshold · traffic
- Rollback trigger: number · watcher · action
- Who signs each step
- Bonus: one CI step to add

Three things to remember

1

Tests for code, tests for data, tests for models.

A pipeline that runs on every change and stops at the first failure. You saw it stop at the data test; nothing was trained, nothing was deployed.

2

Ship the artefact, not the file.

Code, configuration, data reference, model, environment, model card, test scenarios; in a container, so it runs the same everywhere.

3

Every rollout strategy is a way to make a mistake small.

Shadow to prove, canary to limit, A/B to decide, and a rollback trigger written before you start.

Your forum this week: Session 6 · 17–23 October

Q1 Design a CI/CD flow (diagram plus short description) for a fraud-detection model, showing which automated tests run on code and which run on data before deployment.

Q2 Compare canary and blue-green deployments for a customer-facing bank model. Which would you recommend under a strict zero-downtime requirement, and why?

Q3 Describe an A/B test you would run to decide whether a new model version outperforms the current one. Which metric and decision threshold would you use?

Opens today, closes Friday 23 October. Use the IberBank brief on campus for all three. Post early; reply to two classmates.

Before Session 7 (Saturday 24 October): read, and look at one thing

Read

- Treveil et al., Introducing MLOps, chapter 7: Monitoring and feedback loop
- Optional: Huyen, Designing ML Systems, chapter 8: Data distribution shifts and monitoring

Look at one thing

- MODEL_CARD.md in the demo repository says: "Retrain trigger: accuracy below 0.85 on the monthly labelled sample"
- Write down what you would need to measure every month to know whether that line was crossed
- Session 7 starts from there

Next: Session 7 · Monitoring, Explainability & Responsible AI

Data drift, concept drift, performance decay · thresholds that trigger retraining · explanations as an operational requirement · bias. Video 1 (monitoring metrics, with a quiz) opens the same day.

Appendix · The tools (not examinable)

The mechanics, for students who want them, plus a hands-on lab (S05/deploy_lab): the Dockerfile, the four files that make a model a REST API, running it locally and deploying to Cloud Run, a CI workflow file, the complete pipeline automated end to end (trigger → test → train → deploy), pipeline orchestration, and serverless. Not covered live; not in the exam.

Appendix · A Dockerfile is a recipe for the box

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY churn_model.pkl app.py ./
EXPOSE 8080
CMD uvicorn app:app --host 0.0.0.0 \
        --port 8080
```

1

Base image

A slim operating system with Python, the same everywhere.

2

Exact libraries

The versions from requirements.txt: no "it worked on my laptop".

3

Model + code + start command

docker build makes the image; docker run starts it; a registry stores it with a version.

Appendix · Package a model as a REST API: the four files

1. **churn_model.pkl**
   the trained model, saved by train.py
2. **app.py**
   the API: /predict, /health, and auto docs at /docs (FastAPI)
3. **requirements.txt**
   the exact library versions: reproducible
4. **Dockerfile**
   the recipe that packages the four (previous slide)

```python
from fastapi import FastAPI
from pydantic import BaseModel
import pickle, pandas as pd

model = pickle.load(open("churn_model.pkl","rb"))
app = FastAPI()

class Customer(BaseModel):
    tenure_months: int
    monthly_charges: float
    # ...four more features

@app.post("/predict")
def predict(c: Customer):
    X = pd.DataFrame([c.model_dump()])
    p = model.predict_proba(X)[0, 1]
    return {"churn_probability": float(p)}
```

Hands-on: S05/deploy_lab builds and runs this. FastAPI validates the request and generates a docs page; last year's Flask did the same job with more code.

Appendix · Run it on your laptop, then deploy to Google Cloud Run

On your laptop

```
# build the image from the recipe
docker build -t churn-api:v1 .
# run it, map the port, test it
docker run -d -p 8080:8080 churn-api:v1
curl localhost:8080/predict -d @customer.json \
     -H "Content-Type: application/json"
```

To the cloud (Cloud Run)

```
# one command: build + deploy
gcloud run deploy churn-api \
   --source . --region europe-west1 \
   --allow-unauthenticated
# -> https://churn-api-xxxx.run.app
```

Cloud Run: serverless containers

Hands you an HTTPS URL; scales to zero; you pay for the seconds it runs. The simplest way to put a container in the cloud.

GKE (Kubernetes): the scale path

Same image, more moving parts. Worth it when one service becomes many, or needs GPUs. Start on Cloud Run; move when you outgrow it.

The public URL is where Session 3's endpoint protections live: authentication, rate limits, logging. Full walkthrough: S05/deploy_lab/deploy_to_cloud_run.md.

Appendix · A CI workflow file is the pipeline written down

```
name: ci
on: [push, pull_request]
jobs:
  pipeline:
    runs-on: ubuntu-latest
    steps:
      - name: Tests on code
        run: python -m pytest -q
      - name: Validate data
        run: python src/check_data.py
      - name: Train and evaluate
        run: python src/train.py
```

1

When

On every push and pull request. Could also be a schedule or a new-data event.

2

Where

A fresh machine each time: nothing left over from yesterday.

3

What, in order

Named steps; the first failure stops the run. GitLab CI, Jenkins, Azure Pipelines: same idea, different file.

Appendix · The complete pipeline in one file: trigger → test → train → deploy

Three triggers

```yaml
on:
  push: {branches: [main]}
  schedule:
    - cron: "0 3 * * 1"  # weekly
  workflow_dispatch: {}  # button
```

Two jobs: CI + CT, then CD

```yaml
jobs:
  test-train:           # Pipeline CI + Continuous Training
    steps:
      - run: python train.py       # train on latest data
      - run: pytest -q test_api.py  # the gate
      - upload-artifact: churn_model.pkl

  deploy:               # Model CD
    needs: test-train
    if: github.ref == main && DEPLOY_ENABLED
    steps:
      - auth to Google Cloud (repo secret)
      - run: gcloud run deploy churn-api --source .
```

The modern equivalent of the Jenkins trigger demo: one trigger, the whole pipeline runs itself, ending in a deployed service. Runnable: S05/deploy_lab/pipeline.yml (deploy job needs one GCP secret; CI runs without it).

Appendix · Orchestration in one slide: the pipeline as a scheduled graph



A DAG: tasks with dependencies, run and watched by a scheduler

Apache Airflow

The general-purpose workhorse. Write the DAG in Python; a scheduler runs it. Data and ML pipelines everywhere.

Kubeflow Pipelines

ML-specific; each step is a container on Kubernetes. Strong when you already run Kubernetes.

Managed pipelines

Vertex AI, SageMaker, Azure ML Pipelines: the same idea without running the orchestrator yourself.

You do not need one for this course; know the category exists. The rule: a CI tool automates a linear pipeline; an orchestrator manages a pipeline that has become a graph.

Appendix · Deploying a model as a serverless function, in five steps



A container that loads the model and scores one request

The cloud's container registry (ECR, Artifact Registry, ACR)

Lambda / Cloud Functions / Azure Functions, from the image

A sample request in the console; check the answer and the time

A public URL with authentication, rate limits, logging

The last step is where Session 3's endpoint protections live: who may call it, how often, and what gets logged.